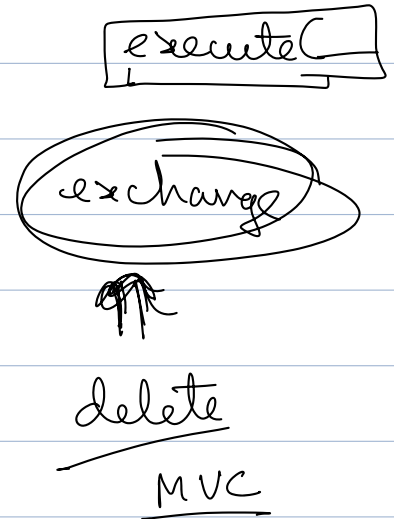


Class will start in 5 minutes

Agenda

- ↳ Intro to Spring Data JPA
- ↳ Repository Pattern
- ↳ Models
-
- ↳ ORM, JPA, Hibernate, JDBC



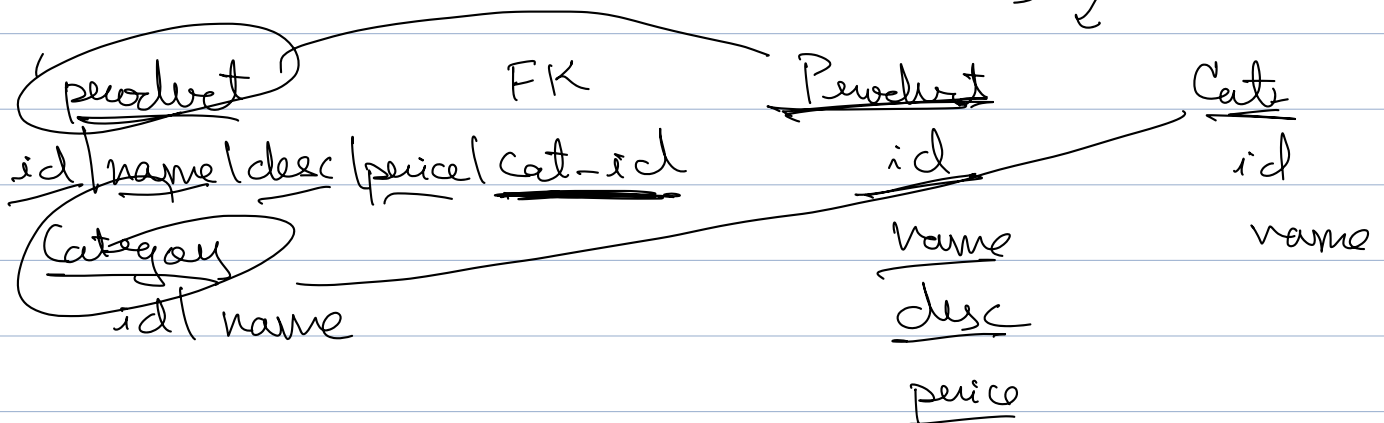
Databases

- ↳ CRUD
- ↳

Worksheet

create table C

class diagram == Schema Diagram



- ① Create tables by writing queries Category
- ② Create tables using the classes that we have automatically

[Model classes have a strong correlation with the DB tables]

ORM Libraries

↳ Object Relational Mapper libraries

↳ Provide a very easy way to work with the database based on the models we have in the codebase.

⇓

① ORM libraries will automatically create corresponding database tables for us using the model classes.

```
fetchAllProducts() {  
    Database db = new Database();  
    db.connect("url", ----)  
    List<Product> rows = db.execute("select * from Product")  
    // Convert  
    // rows objects  
}
```

② ORM helps you to execute queries in the DB as if they were method calls.

```
fetchAllProducts() {  
    List<Product> products = conn.fetchAllProducts()  
    .....  
    save(Product)  
}
```

eg) save(Product)
↳ automatically generate query to save the

⇓
"select * from Product"
⇓
Convert the DB response to Product object

persist in the DB.

and others -

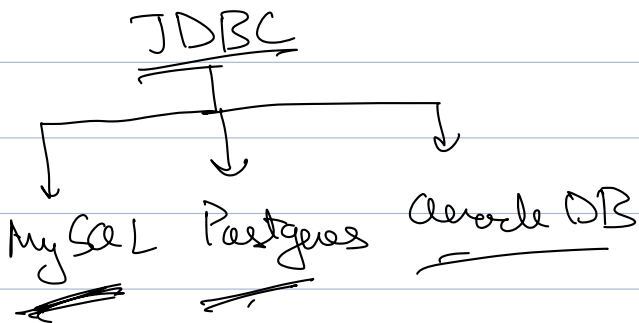
Famous ORMs

↳ Hibernate $\Rightarrow$ default in Spring

↳ MyBatis and Jooq.

JDBC $\Rightarrow$ Java Database Connector

↳ minimal features to interact with the database.



JPA API

↳ Java Persistent API.

↳ A set of suggestions / interface to build ~~Data~~ Java tools to talk to the DB.

Spring Data JPA $\Rightarrow$ A layer on top of JPA that will be helpful in Spring based apps.

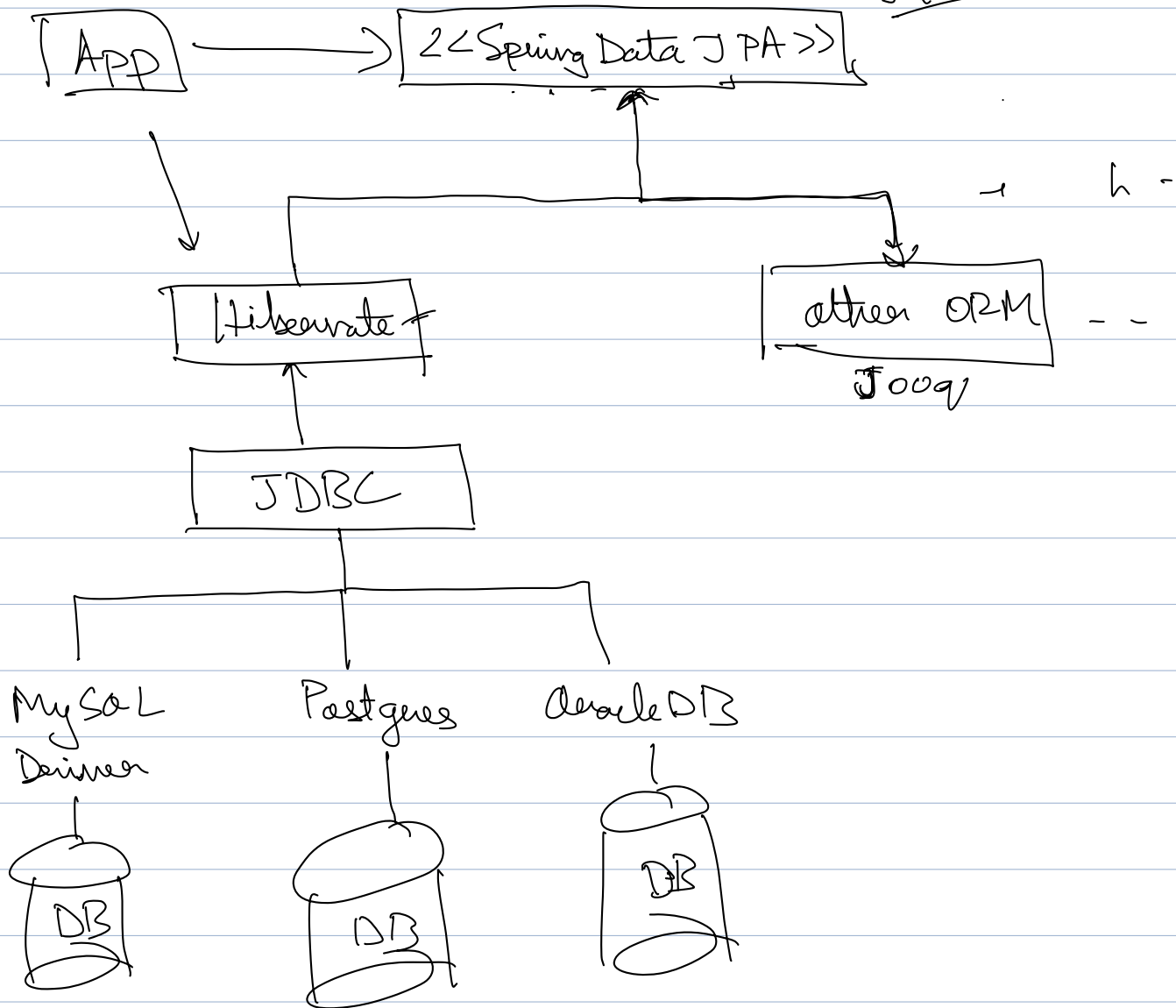
↳ Not an actual implementation $\Rightarrow$ set of interfaces & methods.
CRUD

eg) Repository interface.

↳ Internally uses hibernate for its methods implement

-ation-

→ This is based on JPA

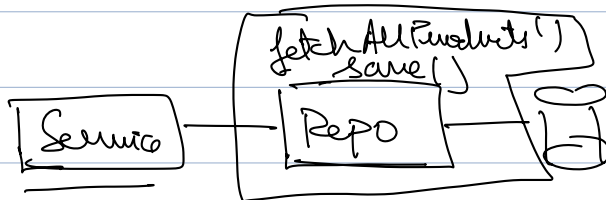


Repository Pattern

⇓

↳ Code to interact with DB should be separate from the business logic;

↳ Service class



Service class does not care how the data is accessed. It only needs the data.

Break for 7 minutes => 10:12 → 10:19

Models

Product (id, title, desc, price, image, category)
Category (id, name)

Common cols

- id
- created_at
- updated_at
- isDeleted

Product

	id	name	desc	price	isDeleted
	1	abc	~	~	0
→	2	bcd	~	20.0	1
	3	cde	~	30.0	

→ Soft delete

① Easier to recover data.

② Deletion takes more time in a DB as compared to update.

class
Base Model {

id
created At
updated At
isDeleted

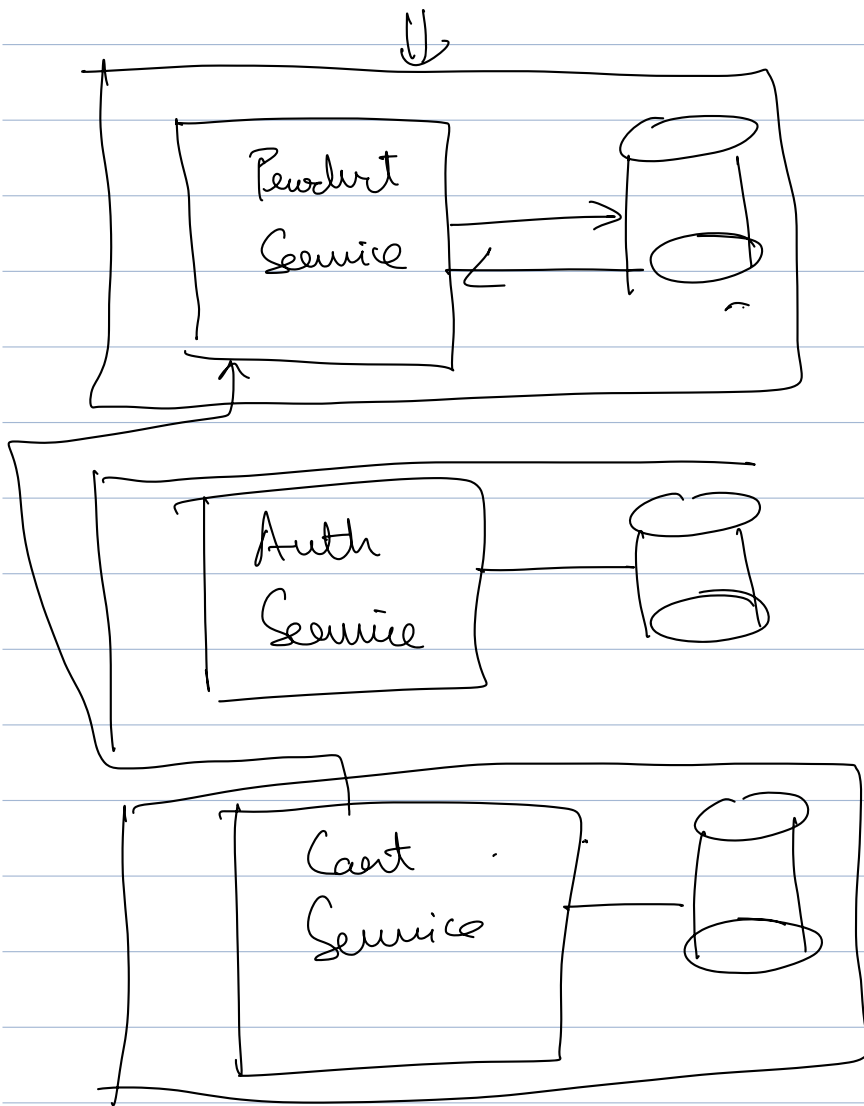
}

class Product extends BaseModel {

name
title
desc

}

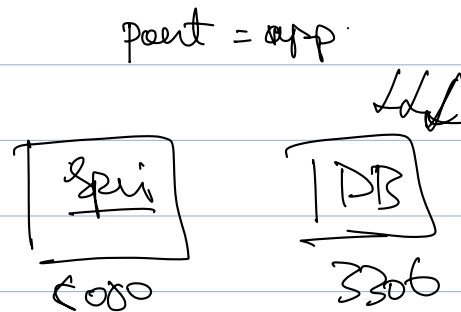
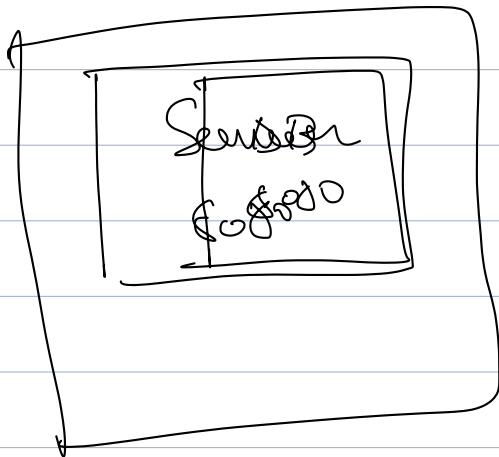
:-



Think of each service as its own company

Each microservice has their own database and one microservice does not directly talk to other microservice's DB.

Doubt



execute → exchange

↳

HTTP Entity

body + headers

null

new HTTPEntityC

new HTTPEntityC